Architecture Decision Records

Tango - YouTube to Anki Vocabulary Pipeline

Version 0.4.0 | July 2026

Introduction

This document records the significant architectural decisions made during the development of Tango up to version 0.4.0. Each record includes the decision, the alternatives that were considered, the reasoning, and the consequences. These records are intended to explain why the system is built the way it is, not just how it works.

ADR-001: genanki over AnkiConnect for card generation

Status: Accepted

Context

AnkiConnect requires Anki desktop to be running and provides an importPackage action that writes to Anki's database directly. genanki produces a portable .apkg file that can be imported at any time without Anki running during generation.

Decision

AnkiConnect for generation was rejected because it ties the generation step to Anki's availability, creates a hard dependency on the local machine's Anki process, and makes automated or server-side generation impossible. genanki's .apkg output is portable, versionable, and inspectable. The tradeoff is that import is a manual step, but this aligns with the tool's design as a pipeline that produces artifacts rather than a live integration.

Consequences

Card generation is fast and Anki-independent. The .apkg file can be imported into any Anki instance including AnkiWeb synced devices. The automated import prompt in the CLI uses importPackage as a convenience but is optional. WSL users cannot use the auto-import due to path translation issues between Linux and Windows paths.

ADR-002: SQLite over a server database

Status: Accepted

Context

The pipeline uses SQLite for all persistent state including definition caches, deck check backlogs, vocabulary records, and run logs.

Decision

A server database such as PostgreSQL was considered but rejected for v0.4.0 because the pipeline is a single-user local tool. SQLite requires no server process, no configuration, and no network access. It is portable as a single file. The WAL journal mode provides sufficient concurrency for the pipeline's sequential access pattern. The decision will be revisited when the pipeline is extended to a multi-user web application.

Consequences

SQLite imposes no practical limits for single-user use. Schema migrations must be handled manually since SQLite does not support ALTER TABLE DROP COLUMN or transactional DDL for all operations. The example_dict2 column migration in v0.4.0 demonstrated the need for explicit migration scripts.

ADR-003: spaCy over NLTK for NLP

Status: Accepted

Context

The pipeline uses spaCy with the en_core_web_sm model for tokenization, lemmatization, and POS tagging.

Decision

NLTK was considered but rejected because its production readiness for tokenization pipelines is lower than spaCy. spaCy provides a single consistent API for all NLP operations, is stateless and serializable, and loads a single model rather than multiple separate resources. The en_core_web_sm model was chosen over en_core_web_md or en_core_web_lg because the accuracy difference for POS tagging specifically is under 4 percent and the speed difference is significant. The decision to upgrade to a larger model is deferred until Phase 3 when semantic similarity tasks require better word vectors.

Consequences

POS tagging on domain-specific vocabulary and non-English text transcribed via auto-generated captions may produce lower accuracy. Single-letter tokens and named entities are not currently filtered, resulting in occasional low-quality cards for proper nouns and abbreviations. These will be addressed in v0.5.0.

ADR-004: Three-condition fuzzy match over pure WRatio

Status: Accepted

Context

Duplicate detection uses WRatio, token_sort_ratio, and a length ratio check rather than WRatio alone.

Decision

WRatio alone was the original approach and produced false positives in production for morphologically rich languages. The word commencer scored 90 percent against comme because WRatio's partial_ratio component finds substring matches regardless of whether one string is semantically related to the other. The three-condition filter requires all of WRatio above the confidence threshold, token_sort_ratio above 50, and a length ratio above 0.6 between the lemma and the matched front. This combination eliminated all observed false positives in French testing while preserving legitimate morphological near-duplicates like faisiez versus fassiez.

Consequences

The filter may still produce false positives in languages with very short root words. The threshold values are configurable via environment variables. The filter is applied only after the short-word and sentence-deck checks, so it does not affect exact matches or sentence-structured decks.

ADR-005: Dual-source definition strategy

Status: Accepted

Context

Example sentences, synonyms, and antonyms are always fetched from dictionaryapi.dev in the native transcript language. Definitions and grammatical class are fetched from the target output language source.

Decision

The original single-source approach fetched everything from the target language API, which meant French vocabulary got English example sentences when DEF_LANG=en. This undermined the pedagogical value of the cards because language learners benefit from seeing words in their native context. The dual-source approach was chosen because dictionaryapi.dev supports native language queries at no additional cost or rate limit. The consequence is that every word in translation mode makes two API calls instead of one, which increases processing time. This is acceptable for v0.4.0 given that the SQLite cache means second and subsequent runs are fast.

Consequences

dictionaryapi.dev coverage varies by language. Words not found in the native API fall back to whatever the target language source provides. When neither source has examples, only

the transcript sentence is shown on the card.

ADR-006: argostranslate over cloud translation APIs

Status: Accepted

Context

The translation module uses community LibreTranslate mirrors first, then a locally installed argostranslate model, rather than a cloud API such as Google Translate or DeepL.

Decision

Google Translate and DeepL were considered but rejected because they require paid API keys for production use volumes. LibreTranslate community mirrors provide free access but have no guaranteed uptime. The argostranslate local model provides reliable offline translation once installed. The 150 megabyte per language pair download is a one-time cost. The latency on CPU without GPU acceleration is the primary drawback, with first-run translation taking up to 60 seconds per word for some language pairs. A 15-second timeout per word was added to prevent silent hangs, and subsequent words in the same session are faster because the model stays loaded.

Consequences

Translation mode is significantly slower than native mode on machines without GPU acceleration. Users are warned before the first local translation and given the option to download the model, continue without translation, or exit. The model path stability across WSL sessions is an open issue.

ADR-007: mono-repo structure for future phases

Status: Proposed

Context

The project will use a mono-repo with separate packages for the pipeline, API server, web application, and browser extension when those phases are built.

Decision

Multi-repo was considered on the grounds that it is standard practice for teams with independent deployment cycles. It was rejected for this project because the components are tightly coupled: the browser extension calls the API server, the API server wraps the pipeline, and all components share domain models for vocabulary and user profiles. A mono-repo allows shared types, a single CI configuration with per-package jobs, and atomic commits when interfaces change across packages. The small team size reinforces this

choice. The mono-repo structure will follow the packages directory convention with one pyproject.toml per package.

Consequences

This decision has not been implemented yet. The current repository contains only the pipeline package. The structure will be established when the API server package is created.